ENES 246 · DIGITAL LOGIC DESIGN

Lecture 14

Adders and Subtractors



$$C_{i+1} = G_i + P_i C_i$$

Half and full adders · ripple-carry adders · carry lookahead · the adder-subtractor · overflow

Reading: Mano & Ciletti, *Digital Design with an Introduction to the Verilog HDL*, 6th ed. — Chapter 4, §4.5

© 2026 Dr. Stefan Cular. All rights reserved.

Today’s roadmap

1 §4.5

Full adders

Half and full, one bit at a time

2 §4.5

Ripple carry

Chain them; count the delay

3 §4.5

Carry lookahead

Generate and propagate

4 §4.5

Subtract, overflow

One circuit, two modes

By the end of class you can…

- Derive the half adder and full adder
- Trace a ripple-carry adder and find its worst-case delay
- Compute generate, propagate and lookahead carries
- Operate the adder-subtractor in both modes
- Detect signed overflow with V = C3 $\oplus$ C4

© 2026 Dr. Stefan Cular. All rights reserved.

Agenda for today's session



Elapsed time	Topic	In-class work
0:00 – 0:05	Welcome, roadmap	—
0:05 – 0:30	Half adder, full adder	Examples 1, 2
0:30 – 0:45	Ripple-carry adder and delay	Example 3 · Your turn 1
0:45 – 0:55	Break	
0:55 – 1:15	Carry generate, propagate, lookahead	Example 4
1:15 – 1:40	Adder-subtractor and overflow	Examples 5, 6 · Your turn 2
1:40 – 1:50	Verilog adders; wrap-up	Lab Hp1 preview

© 2026 Dr. Stefan Cular. All rights reserved.

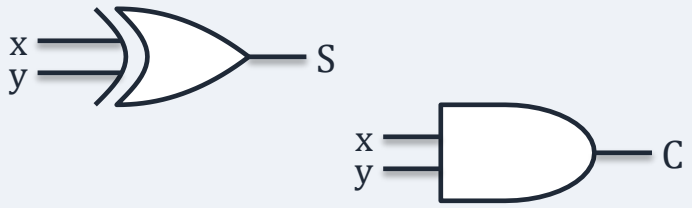
Example 1 — The half adder

§4.5

EXAMPLE

Add two bits x and y. Derive the sum S and carry C.

x	y	C	S	meaning
0	0	0	0	0 + 0 = 0
0	1	0	1	0 + 1 = 1
1	0	0	1	1 + 0 = 1
1	1	1	0	1 + 1 = 10



1. A result from 0 to 2 needs two bits: sum S and carry C
2. Fill the table: $1 + 1 = 10$, so $C = 1$, $S = 0$ on the last row
3. S is 1 on rows 01 and 10: $S = x'y + xy'$
4. That is the XOR: $S = x \oplus y$
5. C is 1 only on row 11: $C = xy$
6. One XOR and one AND

© 2026 Dr. Stefan Cular. All rights reserved.

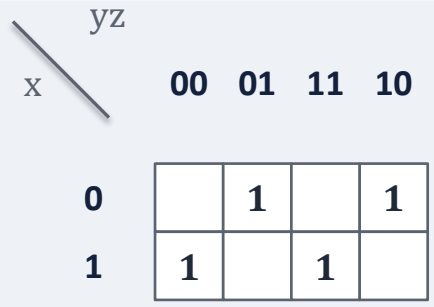
Example 2 — The full adder

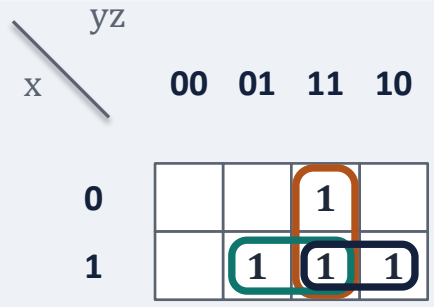
§4.5

EXAMPLE

Add x, y and a carry in z. Derive S and C.

x y z	C	S
0 0 0	0	0
0 0 1	0	1
0 1 0	0	1
0 1 1	1	0
1 0 0	0	1
1 0 1	1	0
1 1 0	1	0
1 1 1	1	1



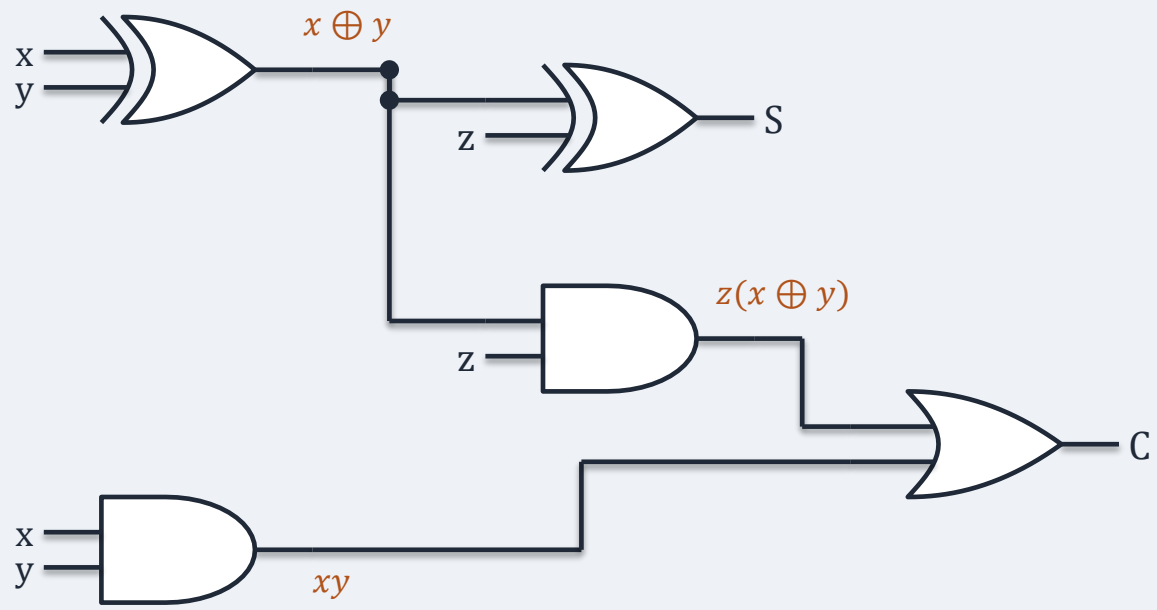


1. Each row: add the three bits, write the result as CS
2. $S = 1$ when the count of 1s is odd: $\Sigma m(1,2,4,7)$
3. S map: a checkerboard, no two 1s adjacent
4. So $S = x \oplus y \oplus z$ (odd function)
5. $C = 1$ when two or more are 1: $\Sigma m(3,5,6,7)$
6. C map: m3, m7 → yz; m5, m7 → xz; m6, m7 → xy
7. $C = xy + xz + yz$; also $C = xy + z(x \oplus y)$

© 2026 Dr. Stefan Cular. All rights reserved.

A full adder from two half adders

§4.5



Why it works

$$S = (x \oplus y) \oplus z$$

$$C = xy + z(x \oplus y)$$

A carry comes from x, y both 1, or from exactly one of them plus z.

Equals $xy + xz + yz$ on every row.

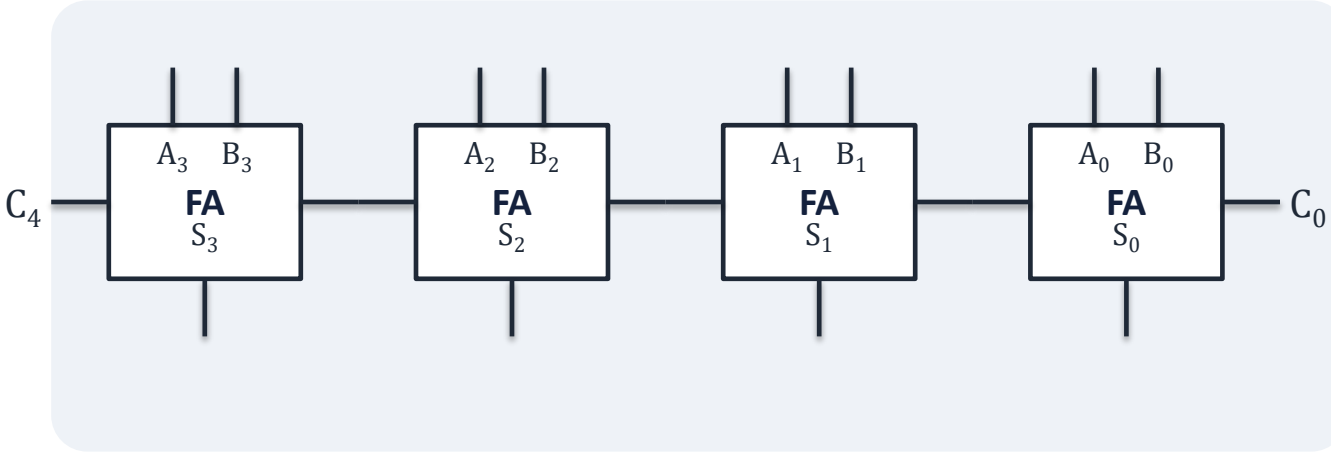
5 gates; the terms $x \oplus y$ and xy reappear as P and G in carry lookahead.

© 2026 Dr. Stefan Cular. All rights reserved.

Example 3 — Ripple-carry adder

§4.5

EXAMPLE Trace $1011 + 0110$ ($C_0 = 0$) through a 4-bit ripple adder. Each full adder has $t_{pd} = 12$ ns.



Stage	A	B	C_i	S	C_{i+1}
0	1	0	0	1	0
1	1	1	0	0	1
2	0	1	1	0	1
3	1	0	1	0	1

1. Stage 0: $1 + 0 + 0 = 01 \rightarrow S_0 = 1$, carry 0
2. Stage 1: $1 + 1 + 0 = 10 \rightarrow S_1 = 0$, carry 1
3. Stage 2: $0 + 1 + 1 = 10 \rightarrow S_2 = 0$, carry 1
4. Stage 3: $1 + 0 + 1 = 10 \rightarrow S_3 = 0, C_4 = 1$
5. Result C_4S = **10001** $= 17 = 11 + 6$ ✓
6. Worst case: the carry ripples through 4 stages
7. $4 \times 12 =$ **48 ns**; $f_{max} = 1/48$ ns $= 20.8$ MHz
8. 16 bits: 192 ns. Delay grows with n

© 2026 Dr. Stefan Cular. All rights reserved.

Your turn 1 — Ripple trace

§4.5

YOUR TURN

≈ 8 min

(a) Trace $1101 + 0111$ with $C_0 = 0$ stage by stage. Give S and C_4.

(b) Check your answer in decimal.

(c) How long does a 32-bit ripple adder take with 12 ns full adders?

Fill in a table: stage, A, B, carry in, S, carry out.

© 2026 Dr. Stefan Cular. All rights reserved.

Your turn 1 — Solution

SOLUTION

Stage	A	B	C_i	S	C_{i+1}
0	1	1	0	0	1
1	0	1	1	0	1
2	1	1	1	1	1
3	1	0	1	0	1

1. Stage 0: $1 + 1 + 0 = 10 \rightarrow S_0 = 0$, carry 1
2. Stage 1: $0 + 1 + 1 = 10 \rightarrow S_1 = 0$, carry 1
3. Stage 2: $1 + 1 + 1 = 11 \rightarrow S_2 = 1$, carry 1
4. Stage 3: $1 + 0 + 1 = 10 \rightarrow S_3 = 0, C_4 = 1$
5. (a) $S =$ **0100**, $C_4 =$ **1**: the 5-bit result 10100
6. (b) $13 + 7 = 20 = 16 + 4 = 10100_2$ ✓
7. (c) $32 \times 12 =$ **384 ns**
8. $f_{max} = 1/384$ ns ≈ 2.6 MHz: far too slow for a CPU

© 2026 Dr. Stefan Cular. All rights reserved.

Break

10 minutes — we resume at the 0:55 mark

Next: carry lookahead and subtraction

© 2026 Dr. Stefan Cular. All rights reserved.

Carry lookahead

§4.5

Generate and propagate

$G_i = A_i B_i$: stage i makes a carry by itself

$P_i = A_i \oplus B_i$: stage i passes an incoming carry on

$$S_i = P_i \oplus C_i, \quad C_{i+1} = G_i + P_i C_i$$

Expand so every carry depends only on P, G and C_0:

Lookahead carries
$C_1 = G_0 + P_0 C_0$
$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$
$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

The payoff

Each carry is a two-level SOP: all carries appear at once, after three gate delays (P and G, then AND-OR), whatever the width.

The price: gates with more inputs as n grows.

© 2026 Dr. Stefan Cular. All rights reserved.

Example 4 — Lookahead by hand

§4.5

EXAMPLE For $A = 1011$, $B = 0011$, $C_0 = 0$ compute P, G, every carry and S.

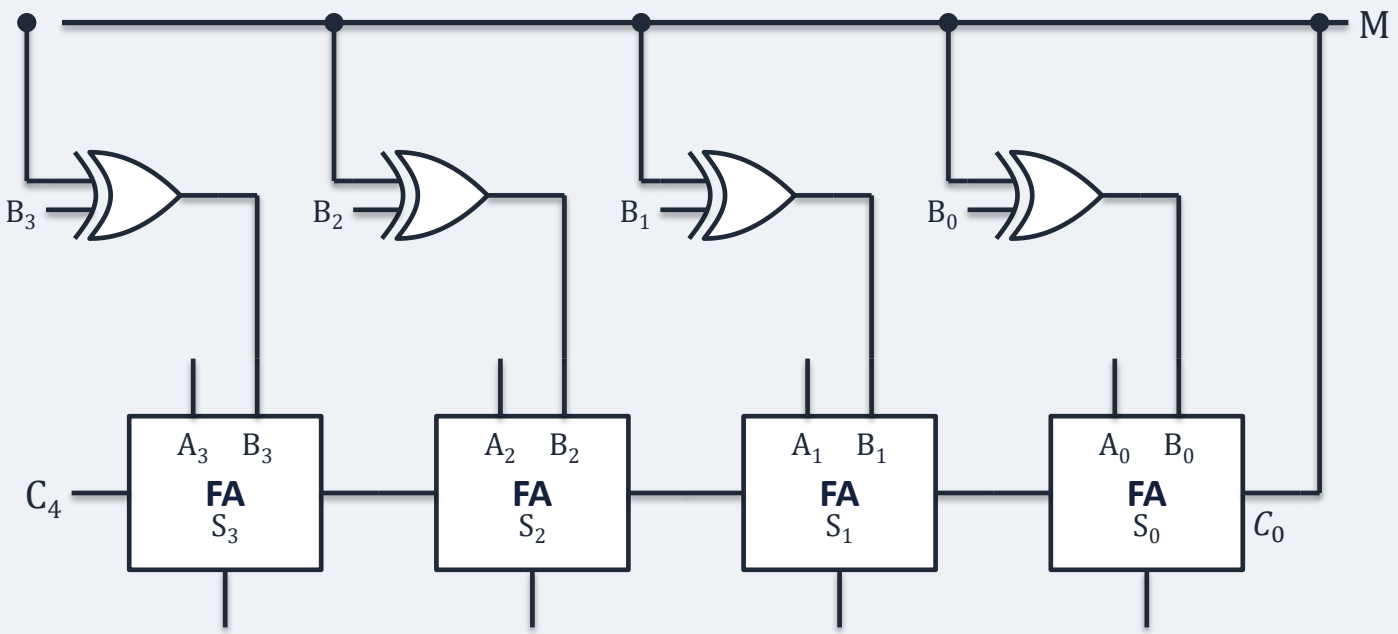
Bit i	3	2	1	0
A	1	0	1	1
B	0	0	1	1
P = A⊕B	1	0	0	0
G = AB	0	0	1	1
C_i	0	1	1	0
S = P⊕C	1	1	1	0

1. $P_i = A_i \oplus B_i$, bits 3–0: 1,0,0,0 → $P = 1000$
2. $G_i = A_i B_i$, bits 3–0: 0,0,1,1 → $G = 0011$
3. $C_1 = G_0 + P_0 C_0 = 1 + 0 = 1$
4. $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0 = 1 + 0 + 0 = 1$
5. $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + \cdots = 0 + 0 + 0 = 0$
6. $C_4 = G_3 + P_3 C_3 = 0 + 1 \cdot 0 = 0$
7. $S_i = P_i \oplus C_i$: $1 \oplus 0,\ \ 0 \oplus 1,\ \ 0 \oplus 1,\ \ 0 \oplus 0$
8. $S = \mathbf{1110} = 14 = 11 + 3$ ✓

© 2026 Dr. Stefan Cular. All rights reserved.

The adder-subtractor

§4.5



One control line

$B \oplus 0 = B$, $B \oplus 1 = B'$: the XOR is a controlled inverter.

$M = 0$: $A + B + 0 = A + B$

$M = 1$: $A + B' + 1 = A - B$ (2's complement)

Overflow (signed)

$V = C_3 \oplus C_4$: carry into and out of the sign bit differ.

Unsigned: C_4 is the carry (add) or no-borrow flag (subtract).

© 2026 Dr. Stefan Cular. All rights reserved.

Example 5 — Both modes

§4.5

EXAMPLE

Trace $A = 0110$, $B = 0011$ through the adder-subtractor with $M = 0$ and then $M = 1$.

M = 0: add

i	A	B⊕M	C_i	S	C_{i+1}
0	0	1	0	1	0
1	1	1	0	0	1
2	1	0	1	0	1
3	0	0	1	1	0

M = 1: subtract

i	A	B⊕M	C_i	S	C_{i+1}
0	0	0	1	1	0
1	1	0	0	1	0
2	1	1	0	0	1
3	0	1	1	0	1

1. $M = 0$: $B \oplus 0 = B = 0011$ and $C_0 = 0$
2. Trace the table: $S = 1001 = 9 = 6 + 3$ ✓
3. $M = 1$: each XOR inverts: $B \oplus 1 = 1100$
4. M also sets $C_0 = 1$: the sum is $A + B' + 1$
5. Trace: $0110 + 1100 + 1 = 1\,0011$
6. Drop C_4: $S =$ **0011** $= 3 = 6 - 3$ ✓
7. $C_4 = 1$ means no borrow: $A \geq B$

© 2026 Dr. Stefan Cular. All rights reserved.

Example 6 — Overflow in four cases

§4.5

Operands	Bits	Result	Reads as	C_3	C_4	V
(+5) + (+2)	0101 + 0010	0111	+7	0	0	0
(+5) + (+4)	0101 + 0100	1001	−7	1	0	1
(−5) + (−4)	1011 + 1100	0111	+7	0	1	1
(+5) + (−4)	0101 + 1100	0001	+1	1	1	0

Reading the table

4-bit signed range is −8 … +7. Rows 2 and 3 fall outside it: the result has the wrong sign, and $C_3 \neq C_4$ flags it.

Operands of opposite sign (row 4) can never overflow.

© 2026 Dr. Stefan Cular. All rights reserved.

Your turn 2 — Subtract and flag

§4.5

YOUR TURN

≈ 10 min

(a) Use the adder-subtractor ($M = 1$) to compute $0100 - 0110$. Give S, C_4 and the value.

(b) Add $(-6) + (-3)$ in 4-bit 2's complement. Find C_3, C_4 and V.

(c) A full adder built from 2-input NAND gates only needs how many gates?

For (c): an XOR costs 4 NANDs; look for a shared gate.

© 2026 Dr. Stefan Cular. All rights reserved.

Your turn 2 — Solution

§4.5

SOLUTION

1. (a) $M = 1$: $B = 0110 \rightarrow B' = 1001$, and $C_0 = 1$
2. Add: $0100 + 1001 + 1 =$ **1110**, $C_4 = 0$
3. $C_4 = 0$: a borrow, so $A < B$: the result is negative
4. 2's comp. of 1110: $0001 + 1 = 0010 = 2 \rightarrow$ **−2** ✓
5. (b) -6: $0110 \rightarrow 1001 + 1 = 1010$; -3: $0011 \rightarrow 1101$
6. Add: $1010 + 1101 = 1\,0111$
7. Bits 0–2: $010 + 101 = 111$, no carry: $C_3 = 0$; sign: $C_4 = 1$
8. $V = C_3 \oplus C_4 =$ **1**: overflow $(-9 < -8)$
9. (c) An XOR from NANDs takes 4 gates; two XORs: 8
10. $C = xy + (x \oplus y)z = [(xy)'((x \oplus y)z)']'$
11. $(xy)'$ and $((x \oplus y)z)'$ are already inside the XORs
12. Only the last NAND is new: $8 + 1 =$ **9** gates

© 2026 Dr. Stefan Cular. All rights reserved.

Adders in Verilog

§4.12

```
module full_adder (input x, y, z,
                   output S, C);
  assign S = x ^ y ^ z;
  assign C = (x & y) | (z & (x ^ y));
endmodule

module add_sub4 (input [3:0] A, B,
                 input M,
                 output [3:0] S,
                 output C4, V);
  wire [3:0] Bx = B ^ {4{M}};
  assign {C4, S} = A + Bx + M;
  assign V = (A[3] ~^ Bx[3]) & (S[3] ^ A[3]);
endmodule
```

Two styles

Structural: instantiate four full_adder modules and wire the carries (Lab Hp2).

Behavioral: {C4, S} = A + Bx + M; the synthesizer picks the adder.

{4{M}} repeats M four times, so every B bit is XORed with M.

V: the operands have the same sign and the result does not (equivalent to $C_3 \oplus C_4$).

© 2026 Dr. Stefan Cular. All rights reserved.

Concept check

1. Why is the half adder called ‘half’?

2. What limits the speed of a ripple-carry adder?

3. What do G_i and P_i mean in words?

4. Why does M feed both the XOR gates and C_0?

© 2026 Dr. Stefan Cular. All rights reserved.

Key takeaways

Half adder

$$S = x \oplus y, C = xy$$

Full adder

$$S = x \oplus y \oplus z$$

Carry

$$C = xy + z(x \oplus y)$$

Lookahead

$$C_{i+1} = G_i + P_i C_i$$

Subtract

$$A + B' + 1$$

Overflow

$$V = C_3 \oplus C_4$$

Four full adders and four XOR gates add and subtract.

Homework #7 due at the start of Lecture 16

This afternoon's lab: Lab G — 4-to-1 multiplexer from basic gates

Next: decimal adders, multipliers and comparators

© 2026 Dr. Stefan Cular. All rights reserved.